

CSCI 6461 | Computer Systems Architecture | TOPIC 3

# The Hardware/Software Interface & AArch64 Programmer's Model

Architectural state, registers, memory organization,  
and programmer-visible conventions

Dr. J. Burns

# Topic 3 Outline

## ① Architectural State & The Hardware Contract

- Concrete definition of architectural state vs. microarchitectural structures.
- The AArch64 programmer's model and processor execution modes.

## ② AArch64 Register File & Naming Conventions

- The 31 general-purpose registers: 64-bit ( $x0$ – $x30$ ) vs. 32-bit ( $w0$ – $w30$ ).
- Special-purpose registers: Zero register ( $xzr/wzr$ ), stack pointer ( $sp$ ), program counter ( $pc$ ).
- Standard ABI register usage: arguments, return values, caller-saved, and callee-saved registers.

## ③ Memory Organization & Endianness

- Byte-addressable 64-bit address space, memory alignment rules, and byte ordering.

## ④ Core Instruction Classes & Execution Mechanics

- Instruction syntax, immediate construction, and the basic load/store model.

## PART 1

# Architectural State & Execution Model

# What Is Architectural State?

- **Definition:** The software-visible processor state needed to describe a program's execution at a particular point.
- On a context switch or exception, saving and later restoring the required architectural state allows execution to resume with the same program-visible behavior.
- **Core Components in AArch64:**
  - General-Purpose Registers (x0–x30).
  - Program Counter (pc) and Stack Pointer (sp).
  - Processor state (pstate), including the architectural location of condition flags.
  - System control and configuration registers (e.g., TTBR0\_EL1, SCTLR\_EL1 for memory management).
  - Architecturally visible memory state associated with the executing program.

## State Checkpointing

Operating systems rely on architectural state to suspend and resume execution while preserving program behavior.

# Architectural vs. Microarchitectural State

## Architectural State (Visible Contract)

- Defined strictly by the ISA specification.
- Software directly reads/writes this state.
- **Examples:** Registers x0–x30, pc, architectural memory and processor state.

## Microarchitectural State (Hidden)

- Internal hardware implementation details.
- Invisible to software correctness.
- **Examples:** Pipeline latches, L1/L2 caches, Branch Target Buffers, rename tables.

## The Execution Boundary

Hardware may execute instructions out of order internally, but retirement must preserve the architectural behavior required by program order.

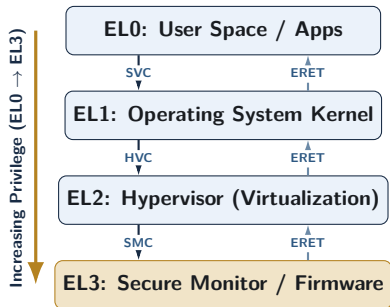
# The AArch64 Execution State

- **AArch64:** The 64-bit execution state introduced in ARMv8-A.
- **64-Bit Address Representation:** General-purpose registers can hold 64-bit pointers, while implemented virtual-address sizes are smaller and configuration-dependent. A64 instructions are fixed 32-bit words aligned to 4-byte boundaries.
- **Distinct A64 Instruction Set:** A64 defines a separate encoding model from A32, removes broad per-instruction conditional execution, and keeps the Program Counter (pc) outside the general-purpose register file.

## A64 Design Consequence

A64 is a distinct instruction set from A32, allowing ARM to simplify instruction encoding and remove several older architectural conventions while preserving compatibility through separate execution states on implementations that support them.

# Privilege Levels (Exception Levels)



## CSCI 6461 Scope

Exception levels are included only for architectural context. CSCI 6461 focuses on the programmer-visible ISA, registers, instructions, addressing, and control flow.

- **EL0 (Application):** Unprivileged user space executing application logic with private virtual address spaces.
- **EL1 (OS Kernel):** Privileged kernel execution managing memory translation, drivers, and process scheduling
- **EL2 (Hypervisor):** Non-secure virtualization layer enforcing second-stage address translation across guest OS instances
- **EL3 (Secure Monitor):** Secure monitor / firmware execution level used for selected security-state transitions and platform firmware

PART 2

# AArch64 Register Architecture & ABI



# General-Purpose Registers: x0–x30

- AArch64 provides **31 general-purpose registers** (x0–x30), each exactly **64 bits wide** (8 bytes).
- **Highly Regular Design:** Most instructions allow general-purpose registers to be used uniformly as sources and destinations, with specific exceptions depending on the instruction encoding.
- **Regular Operand Model:** AArch64 avoids many legacy implicit-register conventions, although some instructions still assign special roles to particular registers.

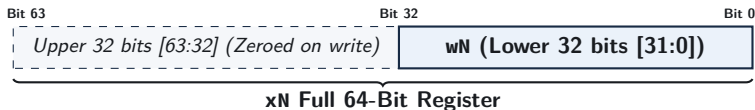


## Why Exactly 31 Registers?

Many A64 register fields are 5 bits wide ( $2^5 = 32$  encodings). Encoding value 31 denotes either the zero register (xZR/wZR) or the stack pointer (SP/WSP) depending on the instruction class.

## 32-Bit Register Views: $w_0$ – $w_{30}$

- Every 64-bit  $x$  register has a corresponding 32-bit view denoted as  $w_0$  through  $w_{30}$ .
- $w_N$  represents the **lower 32 bits (bits 31:0)** of the physical register  $x_N$ .
- Enables native 32-bit integer arithmetic without requiring separate hardware register files.



### Zero-Extension Safety Rule

Writing to a 32-bit  $w$  register automatically zeroes bits 63:32 of the corresponding  $x$  register.

# The Zero Register: `xzr` and `wzr`

- Encoded as register number 31 within standard ALU instruction encodings.
- **Read Behavior:** Always returns constant zero (0 for `xzr`, 32-bit 0 for `wzr`).
- **Write Behavior:** Values written to it are discarded.
- **Architectural Benefit:** Lets several common operations reuse existing instruction formats and supports convenient aliases such as `cmp`.

## Common Idioms Using the Zero Register

| Assembly Instruction          | Equivalent Operation                          | Purpose                                                   |
|-------------------------------|-----------------------------------------------|-----------------------------------------------------------|
| <code>mov x0, xzr</code>      | $x_0 \leftarrow 0$                            | Clear register $x_0$ to zero                              |
| <code>subs xzr, x1, x2</code> | Evaluate $x_1 - x_2 \rightarrow \text{flags}$ | Compare $x_1$ and $x_2$ (alias: <code>cmp x1, x2</code> ) |
| <code>str xzr, [x1]</code>    | $\text{Mem}[x_1] \leftarrow 0$                | Zero out a 64-bit memory word                             |

# Special-Purpose Registers: `sp`, `pc`, `lr`

## Stack Pointer (`sp`)

- Dedicated register holding stack top address.
- Must maintain **16-byte alignment** in the ABI.
- Primarily tracks the current stack location; selected arithmetic and load/store instructions permit `sp` as an operand.

## Program Counter (`pc`)

- Represents the address associated with current instruction execution.
- **Cannot** be read/written as data (no `mov x0, pc`).
- PC-relative addresses are formed with instructions such as `adr` and page-relative `adrp`.
- Modified via branch/call instructions.

## Link Register (`x30` / `lr`)

`x30` is a general-purpose register conventionally used as the link register. `bl` and `blr` place the return address in `x30`; `ret` normally returns through it.

# The Calling Convention: Caller- vs. Callee-Saved

## Caller-Saved Registers (x0–x17)

- Used for arguments and temporary scratch math.
- Caller **must save them** to stack before calls if values are needed later.
- The ABI does not require the callee to preserve their incoming values.

## Callee-Saved Registers (x19–x28)

- Preserves long-lived variables across calls.
- If callee modifies these, it **must save and restore** their original contents.
- Guarantees protection of caller state.

## The Efficiency Trade-off

Balancing caller- and callee-saved registers across the ABI minimizes memory traffic and stack spilling during nested subroutine calls.

# AArch64 Register File Overview

| Register | Architectural Role                                     | Preserved Across Calls? (ABI) |
|----------|--------------------------------------------------------|-------------------------------|
| x0–x7    | Function arguments & return values                     | No (Caller-saved / Scratch)   |
| x8       | Indirect result location (struct return pointer)       | No (Caller-saved)             |
| x9–x15   | Temporary / Scratch registers                          | No (Caller-saved)             |
| x16–x17  | Intra-procedure-call temporary registers (IP0, IP1)    | No (Caller-saved)             |
| x18      | Platform register (reserved for OS / TLS)              | Platform-dependent            |
| x19–x28  | Callee-saved registers                                 | <b>Yes (Callee-saved)</b>     |
| x29 (fp) | Frame pointer (base of current stack frame)            | <b>Yes (Callee-saved)</b>     |
| x30 (lr) | Link register (subroutine return address)              | No (Caller-saved)             |
| sp       | Stack pointer; ABI alignment must be maintained        | ABI invariant                 |
| xzr      | Dedicated zero register (constant 0, writes discarded) | N/A (Hardware constant)       |

# Process State: `pstate`

- `pstate` contains selected processor status fields visible to the architecture.
- One part of `pstate` holds the condition flags used by conditional execution.
- Topic 4 explains those flags when they are used for comparisons, branches, and conditional select.

## For This Topic

Here, `pstate` is part of the programmer-visible state. Topic 4 explains how N, Z, C, and V are used in assembly control flow.

## PART 3

# Memory Model, Alignment & Endianness



# Memory Organization & Byte Addressing

- Memory is byte-addressed and exposed to software as a sequence of **8-bit bytes**.
- Addresses are represented in 64-bit registers, although implemented virtual-address ranges are smaller than the full 64-bit space.
- Data operands span consecutive byte addresses scaling by standard powers of two:

| Data Type  | Size (Bytes) | Size (Bits) | AArch64 Architecture Term     |
|------------|--------------|-------------|-------------------------------|
| Byte       | 1            | 8-bit       | Byte (b)                      |
| Halfword   | 2            | 16-bit      | Halfword (h)                  |
| Word       | 4            | 32-bit      | Word (w)                      |
| Doubleword | 8            | 64-bit      | Doubleword (x)                |
| Quadword   | 16           | 128-bit     | Quadword / Vector element (q) |

## 64-Bit Pointer Translation

While pointers are 64 bits wide, implemented virtual and physical address sizes are smaller and configuration-dependent; common systems use ranges such as 48 or 52 bits.

# Memory Alignment Rules

- An access to an  $n$ -byte operand is **naturally aligned** if  $\text{Address} \pmod n = 0$ .
- **Alignment Rules:** Word (4B boundary), Doubleword (8B boundary), Instruction fetch (4B boundary).

## Aligned Access (Single Transaction)

64-bit Word [Bytes 0–7]

## Unaligned Access (Crosses Boundary)

Word Part 1

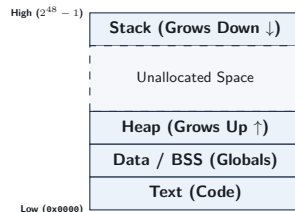
Word Part 2

## Alignment Penalties

Many ordinary accesses to Normal memory can be unaligned, although an unaligned access may require additional memory transactions or cycles. Some access types, memory attributes, or configured environments require alignment and can generate an alignment fault.

# The Process Address Space Layout

- **Text (Code):** Read-only machine instructions.
- **Data / BSS:** Globally accessible and static variables.
- **Heap:** Dynamically allocated memory via malloc; grows **upward** toward higher addresses.
- **Stack:** Local function variables and saved caller state; conventionally grows **downward** toward lower addresses.



## Simplified Address-Space Model

Modern processes also contain shared libraries, memory-mapped files, anonymous mappings, guard pages, and ASLR-created gaps. Excessive recursion typically reaches a stack limit or guard page and faults.

## PART 4

# Foundational Instruction Classes

# Instruction Syntax Conventions

- Many AArch64 data-processing instructions use the form: mnemonic dst, src1, src2\_or\_imm
- Three-Operand Format:** Computation preserves source registers without overwriting them.

| Assembly Example | RTL Operation              | Description                                      |
|------------------|----------------------------|--------------------------------------------------|
| add x0, x1, x2   | $x_0 \leftarrow x_1 + x_2$ | 64-bit register addition                         |
| add w0, w1, w2   | $w_0 \leftarrow w_1 + w_2$ | 32-bit addition (clears upper 32 bits of $x_0$ ) |
| add x0, x1, #16  | $x_0 \leftarrow x_1 + 16$  | Addition with immediate constant                 |

## Explicit Destination Operand

Many AArch64 data-processing instructions use a separate destination register, so source registers need not be overwritten. This differs from traditional two-operand scalar x86 forms, while newer x86 vector extensions also support three-operand forms.

# Loading Immediate Constants: `movz`, `movk`, `movn`

- A fixed 32-bit instruction cannot fit an arbitrary 64-bit constant.
- AArch64 constructs large constants in 16-bit chunks using **wide-immediate instructions**:

| Instruction                      | Operation Name      | Semantic Action                                    |
|----------------------------------|---------------------|----------------------------------------------------|
| <code>movz xd, imm, lsl s</code> | Move Wide with Zero | Places 16-bit immediate at shift $s$ ; others zero |
| <code>movk xd, imm, lsl s</code> | Move Wide with Keep | Replaces 16 bits at shift $s$ ; preserves others   |
| <code>movn xd, imm, lsl s</code> | Move Wide with NOT  | NOT of shifted 16-bit immediate forms destination  |

## Constructing a 32-bit Value (`0x1234abcd`)

|                                        |                             |
|----------------------------------------|-----------------------------|
| <code>movz x0, #0xabcd</code>          | $x_0 = 0x0000000000000abcd$ |
| <code>movk x0, #0x1234, lsl #16</code> | $x_0 = 0x000000001234abcd$  |

# Memory Access: Basic Load/Store Architecture

- Ordinary integer memory access uses explicit load and store instructions such as `ldr` and `str`.
- The instruction form and destination/source register width determine the transfer size:

| Instruction                | Register         | Transfer Width       | Action                                             |
|----------------------------|------------------|----------------------|----------------------------------------------------|
| <code>ldr xt, [xn]</code>  | 64-bit ( $x_t$ ) | 8 Bytes (Doubleword) | $x_t \leftarrow \text{Mem}_8[x_n]$                 |
| <code>str xt, [xn]</code>  | 64-bit ( $x_t$ ) | 8 Bytes (Doubleword) | $\text{Mem}_8[x_n] \leftarrow x_t$                 |
| <code>ldr wt, [xn]</code>  | 32-bit ( $w_t$ ) | 4 Bytes (Word)       | $w_t \leftarrow \text{Mem}_4[x_n]$ (Zero-extended) |
| <code>str wt, [xn]</code>  | 32-bit ( $w_t$ ) | 4 Bytes (Word)       | $\text{Mem}_4[x_n] \leftarrow w_t$                 |
| <code>ldrb wt, [xn]</code> | 8-bit ( $w_t$ )  | 1 Byte               | $w_t \leftarrow \text{Mem}_1[x_n]$ (Zero-extended) |
| <code>strb wt, [xn]</code> | 8-bit ( $w_t$ )  | 1 Byte               | $\text{Mem}_1[x_n] \leftarrow w_t[7 : 0]$          |

## Pointer Referencing

The base register ( $x_n$ ) holds the 64-bit memory pointer, explicitly enclosed in brackets `[xn]` to denote architectural indirection.

**SUMMARY & ASSESSMENT**

# **Review and Self-Test Questions**



## Topic 3: Comprehensive Summary

- **Architectural State:** The programmer-visible contract includes general registers (x0–x30), processor state, stack pointer (sp), program counter (pc), and byte-addressed memory.
- **Register Structure:** The 64-bit x0–x30 registers have corresponding 32-bit w0–w30 views; writing a w register clears the upper 32 bits of the corresponding x register.
- **ABI Roles:** Register use is constrained by calling-convention rules, including argument/return registers and caller- vs. callee-saved registers.
- **Memory Model:** AArch64 exposes byte-addressed memory with alignment requirements and explicit load/store instructions.
- **Instruction Conventions:** Assembly syntax, immediate construction, and basic load/store notation prepare for the detailed addressing-mode treatment in Topic 5.

# Self-Test Questions: State & Register Mechanics

- ❶ **Architectural vs. Microarchitectural State:** Why are CPU branch prediction tables and physical register rename structures classified as microarchitectural state rather than architectural state?
- ❷ **32-bit Register Write Semantics:** Suppose register x5 initially contains `0xffffffff_ffffffff`. If the instruction `mov w5, #0x1234` is executed, what exact 64-bit value resides in x5 post-execution?
- ❸ **Zero Register Application:** Why does AArch64 provide a dedicated zero register (`xzr/wzr`) instead of permanently assigning a general-purpose register (like x0) to hold zero?
- ❹ **Calling Convention Responsibility:** If function `foo()` calls `bar()`, and `foo()` needs to preserve values across the call in registers x3 and x20, which function is responsible for saving each register to the stack?

# Self-Test Questions: Memory & Instruction Mechanics

- ① **Memory Model:** Why does AArch64 use explicit load/store instructions rather than allowing most arithmetic instructions to read directly from memory?
- ② **Alignment:** Why can naturally aligned accesses be simpler for hardware than accesses that cross an alignment boundary?
- ③ **Immediate Construction:** Why may a 64-bit constant require more than one AArch64 instruction using `movz/movk`?

# Looking Ahead

## Next Lecture Preview

### Our Next Topic: AArch64 Assembly Programming: Arithmetic, Logic, & Control Flow

- **ALU & Shifted Operands:** Arithmetic and shifts using AArch64 data-processing instructions.
- **Bitwise Logic & Field Extraction:** Manipulating bitfields using masks, bit clears (`bic`), and bitfield extracts (`ubfx`).
- **Condition Flags & Comparisons:** Using N, Z, C, and V for comparisons and conditional control flow.
- **Control Flow & Conditional Selection:** Translating loops and nested conditionals while using `csel` to reduce reliance on conditional branches for suitable expressions.